

Examen 2020/21-2

Asignatura	Código	Fecha	Hora inicio
Análisis y diseño con patrones	75.586	19/6/2021	09:30

Este enunciado también corresponde a las siguientes asignaturas:

- 76.547 - Análisis y diseño con patrones

Ficha técnica del examen

- Comprueba que el código y el nombre de la asignatura corresponden a la asignatura de la que te has matriculado.
- Tiempo total: **2 horas** Valor de cada pregunta: **Indicado en cada pregunta**
- ¿Puede consultarse algún material durante el examen? **No** ¿Qué materiales están permitidos? **Ninguno**
- ¿Puede utilizarse calculadora? **No** ¿De qué tipo? **Ninguna**
- Si hay preguntas tipo test, ¿descuentan las respuestas erróneas? **Si** ¿Cuánto? **Indicado en la pregunta**
- Indicaciones específicas para la realización de este examen:
- **No es necesario que te identifiques con tu nombre o el número del carné de estudiante. La autoría de la prueba es detectada por el propio sistema.**
- **Responde las preguntas en un documento aparte referenciando claramente la pregunta que estás respondiendo.**
- **No puedes utilizar recursos externos.**
- **En caso de que tengas una duda que no puedes resolver por la vía establecida, haz los supuestos que consideres oportunos, indícalos en tu respuesta, y arguméntalos.**
- **Esta prueba se debe resolver de manera individual bajo tu responsabilidad. En caso de que no sea así, se evaluará con un cero. Por otra parte, y siempre a criterio de los Estudios, el incumplimiento de este compromiso, puede suponer la apertura de un expediente disciplinario con posibles sanciones.**
- **La prueba se puede resolver directamente en el ordenador haciendo uso de una herramienta CASE (e.g., MagicDraw) o a mano. En tal caso:**
 - **No es necesario imprimir el enunciado, puedes responder las preguntas en una hoja en blanco.**
 - **Utiliza un bolígrafo de tinta azul o negra.**

Examen 2020/21-2

Asignatura	Código	Fecha	Hora inicio
Análisis y diseño con patrones	75.586	19/6/2021	09:30

- **Digitaliza tus respuestas en un único archivo en formato PDF o Word. Puedes hacerlo con un escáner o con un dispositivo móvil. Comprueba que la digitalización de tus respuestas es nítida y fácilmente legible.**
-

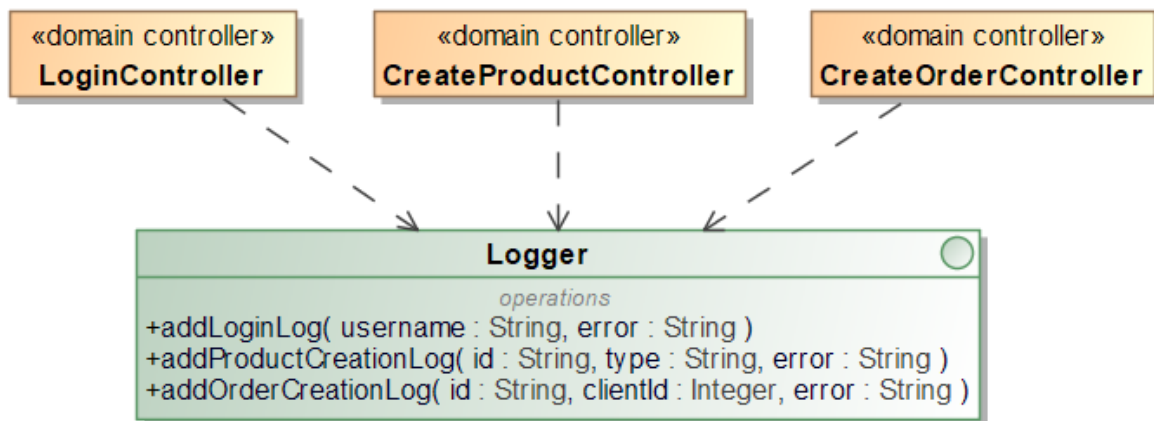
Examen 2020/21-2

Asignatura	Código	Fecha	Hora inicio
Análisis y diseño con patrones	75.586	19/6/2021	09:30

Enunciados con soluciones

Pregunta 1 (15%)

En nuestro negocio los controladores generan logs para almacenar las incidencias de servicio. El servicio de log queda desacoplado de los controladores a través de una interfaz. Identifica qué principio de diseño se incumple en el siguiente diagrama, justifica tu respuesta explicando brevemente cuál sería el diseño correcto:



Solución:

Segregación de Interfaces (ISP). La interfaz Logger ofrece al resto de clases de sistema un gran número de métodos, pero cada clase necesita únicamente el método específico para su dominio (identificación, productos u órdenes de compra).

Separaremos los servicios de esta interfaz en 3 interfaces más cohesivas, una por controlador. Accediendo así cada controlador únicamente al servicio que necesita.

Pregunta 2 (15%)

En el departamento de supervisión de sistemas disponemos de un servicio de mensajería electrónica para la gestión de incidencias. Los controladores de dominio asociados a dicho servicio utilizan su método *send* para el envío de incidencias. El método *send* presenta la siguiente definición:

```

EmailServer::send(String address, String subject, String text, Integer
priority, Boolean confirmation): Boolean
  
```

Examen 2020/21-2

Asignatura	Código	Fecha	Hora inicio
Análisis y diseño con patrones	75.586	19/6/2021	09:30

Ante un problema grave en nuestro servicio de mensajería nos vemos obligados a utilizar temporalmente el servicio de correo de otro departamento. Pero éste ofrece un método de envío con menor número de posibilidades, concretamente su método *sendEmail* tiene la siguiente definición:

```
EMailServer::sendEmail(String address, String subject, String text):  
Boolean
```

No queremos modificar nuestros controladores de dominio para utilizar el nuevo servicio de correo. Justifica cuál de los siguientes patrones es el más adecuado para resolver el problema planteado: ¿**Adapter** o **Decorator**?

Solución:

Adapter (adaptador). Porque necesitamos acceder a una clase a través de operaciones diferentes a las que ofrece originalmente. No estamos añadiendo nuevas funcionalidades con la misma interfaz pública, que es el problema que resuelve el patrón decorator.

Pregunta 3 (10%)

Responde si son ciertas o falsas las siguientes afirmaciones y justifica la respuesta (en un máximo de 3 líneas). Cada respuesta suma 2,5% si se acierta y resta 2,5% si se falla. Las respuestas en blanco o sin justificar no suman ni restan puntos. La nota mínima de esta pregunta será 0.

- a) Las restricciones de integridad sirven para recalcar las reglas más importantes del diagrama estático de análisis.
Falso. Las restricciones de integridad sirven para expresar las reglas que el diagrama estático de análisis no puede representar gráficamente.
- b) El alto acoplamiento sólo se da cuando las clases tienen relaciones innecesarias.
Falso. Un alto acoplamiento también puede suceder si una clase depende mucho de otra. Ej: al cambiar funcionalidades de una clase, la otra clase se ve afectada.
- c) En el patrón de diseño observador, el sujeto siempre se tiene que enviar a sí mismo en la operación notificar.

Examen 2020/21-2

Asignatura	Código	Fecha	Hora inicio
Análisis y diseño con patrones	75.586	19/6/2021	09:30

Falso. Puede enviar los cambios concretos en vez de toda su instancia o incluso puede que sea el observador quien se encargue de pedir el nuevo estado al sujeto.

- d) Cuando queremos representar un rango de fechas siempre será mejor usar el patrón rango que el patrón asociación histórica.

Falso. Dependerá del contexto y de lo que queramos representar con ese rango de fechas. Si buscamos principalmente tener un histórico y no solo datos de un rango se priorizará el patrón asociación histórica.

Pregunta 4 (30%)

Una empresa nos ha pedido que diseñemos un sistema de gestión de jardinería. Este sistema tiene que permitir definir los distintos tipos de productos: plantas, flores, arbustos, árboles y combinación. Una combinación dispone de varios tipos de productos para hacer un proyecto concreto. Además los árboles tienen que registrar la altura mínima y máxima que pueden llegar a tener. Para simplificar, la altura siempre vendrá dada en cm.

Por ejemplo, queremos un conjunto para nuestro jardín que contiene tres tipos de flores, dos arbustos y un árbol que está categorizado con una altura mínima de 100 cm a una máxima de 200 cm.

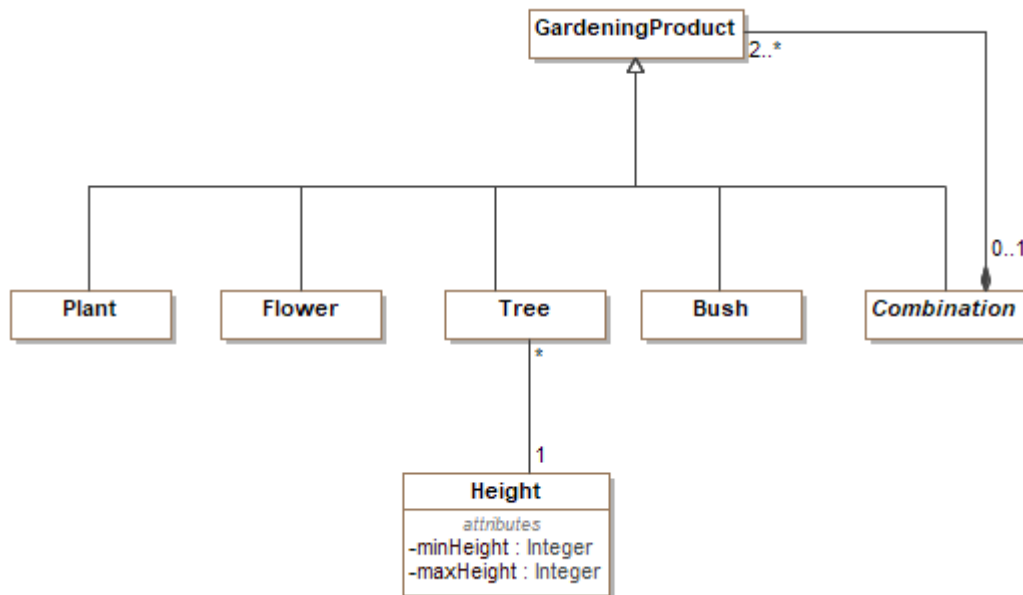
Se pide el diagrama estático de análisis para registrar esta información y las restricciones de integridad. En caso de que hayas aplicado algún patrón de análisis, indica cuál o cuáles has aplicado. Haz los supuestos que creas necesarios y justifica tus decisiones (en unas 5 líneas).

Solución:

Aplicamos el patrón Composite para poder expresar la combinación que como mínimo tendrá dos productos y la hacemos abstracta ya que no tendrá instancias como tal, sino es una que es agrupación. Para la altura del árbol usaremos el patrón Range para marcar la mínima y la máxima.

Examen 2020/21-2

Asignatura	Código	Fecha	Hora inicio
Análisis y diseño con patrones	75.586	19/6/2021	09:30



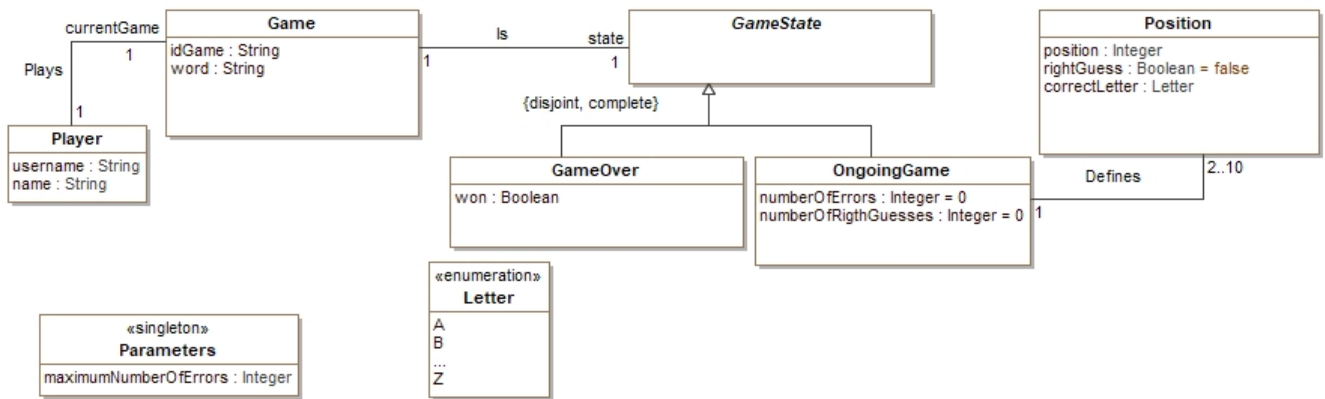
Pregunta 5 (30%)

Queremos diseñar un juego para adivinar palabras. Cada partida permite a un jugador adivinar una palabra. El sistema registra sólo la partida actual de los jugadores. De las partidas se registra su identificador, la palabra a adivinar (que es una palabra entre 2 y 10 letras generada automáticamente por el sistema) y su estado que puede ser en juego o acabada. Las partidas que están en juego guardan el número de errores y el número de aciertos de la partida y tienen asociadas tantas posiciones como número de letras tiene la palabra.

Las posiciones disponen de la letra correcta y de su posición (dentro de la palabra) así como un atributo que indica si se ha adivinado poniendo una letra en aquella posición (es decir, la letra indicada por el jugador coincide con la letra correcta). De las partidas acabadas sólo se registra si se han ganado o perdido. El sistema también registra en su clase *Parameters* el número máximo de errores permitidos. Si un jugador supera este número, habrá perdido la partida. En cambio, si no lo supera y ha adivinado todas las letras de la palabra, entonces habrá ganado. A continuación disponéis del diagrama de clases del sistema:

Examen 2020/21-2

Asignatura	Código	Fecha	Hora inicio
Análisis y diseño con patrones	75.586	19/6/2021	09:30



Restricciones Textuales:

- Una partida se identifica por *idGame*.
- Una posición se identifica por *idGame*, *position*.
- El atributo *correctLetter* de la clase *Position* tiene como valor la letra de la palabra que indica la posición.
- El atributo *position* de la clase *Position* es un valor entre 1 y el número de caracteres de la palabra *word* de la partida.
- El *numberOfErrors* de una partida tiene que ser más pequeño que el *maximumNumberOfErrors* de la clase *Parameters*.
- El *numberOfRightGuesses* es igual al número de posiciones que tienen el atributo *rightGuess* a cierto.
- Todos los enteros son positivos.

a. Proponed el diagrama de secuencia o escribid el pseudocódigo de la operación *Game::play(pos: Integer, letter: Char)*. Esta operación permite al jugador poner una letra en una posición. Podéis asumir (quiere decir que no hace falta que comprobéis) que se cumplen las siguientes condiciones:

- La posición *pos* será un valor que estará entre 1 y el número de letras de la palabra.
- La posición *pos* no tiene ya la letra correcta (*rightGuess* = false).
- La letra *letter* tiene un carácter que es igual a uno de los literales de la enumeración *Letter*.

La operación tiene que hacer lo siguiente:

- Si la partida está en juego:

Examen 2020/21-2

Asignatura	Código	Fecha	Hora inicio
Análisis y diseño con patrones	75.586	19/6/2021	09:30

- Si la letra *letter* coincide con la *correctLetter* de la posición *pos* entonces *rightGuess* toma el valor cierto y se incrementa en 1 el *numberOfRightGuesses* de la partida en juego. En caso contrario, se incrementa en 1 el *numberOfErrors*.
- Si la letra *letter* no coincide con la *correctLetter* de la posición *pos* y el *numberOfErrors* de la partida es igual al *maximumNumberOfErrors* entonces la partida pasa a estar acabada y el atributo *won* toma el valor false.
- Si con la letra *letter* introducida en la posición *pos* se ha acertado la palabra (se han acertado todas las letras de la palabra) entonces la partida pasa a estar acabada y el atributo *won* toma el valor cierto.
- Si la partida está acabada:
 - No se tiene que hacer nada.

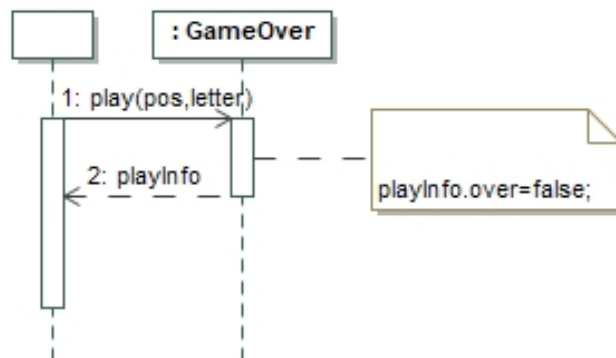
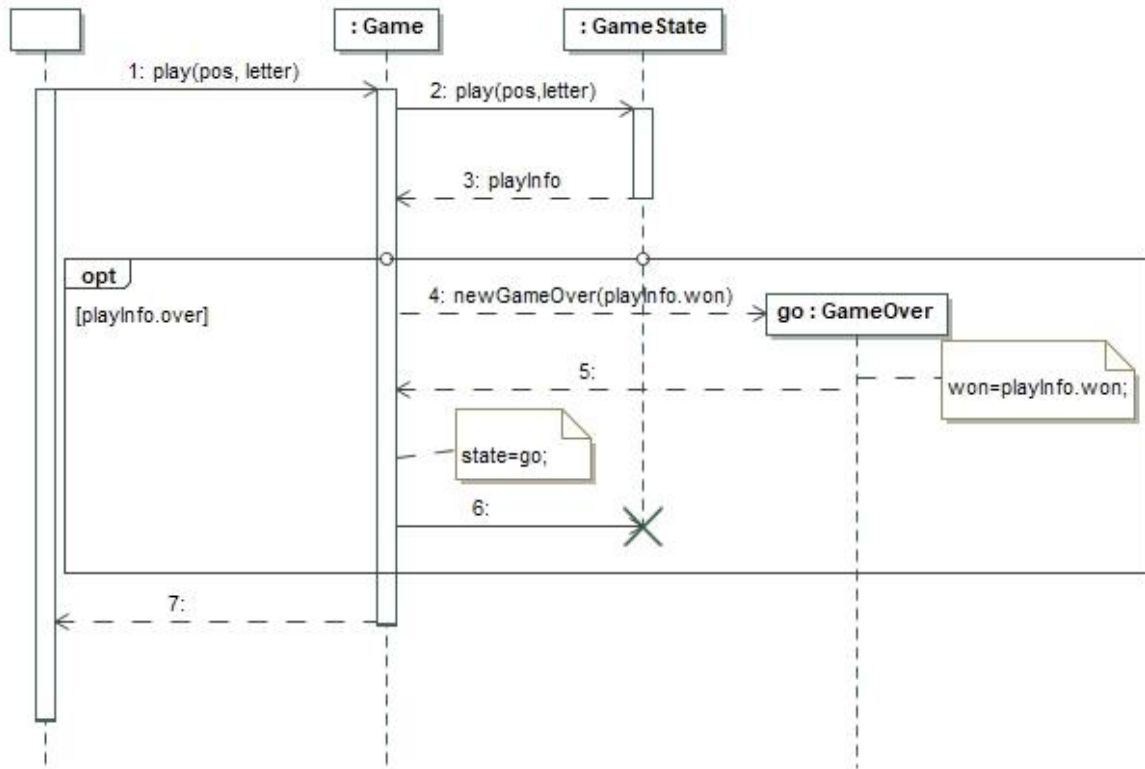
Nota: Podéis suponer la existencia de la operación *String::size(): Integer* que devuelve el número de caracteres que tiene el String *self*.

b. ¿Qué patrones se han aplicado en el diagrama de clases de diseño y en el diseño de la operación del apartado anterior? Tenéis que indicar tanto los patrones de análisis, los de diseño y los de asignación de responsabilidades.

- a. A continuación disponéis de los diagramas de secuencia de la operación (el loop también se puede hacer con un iterador).

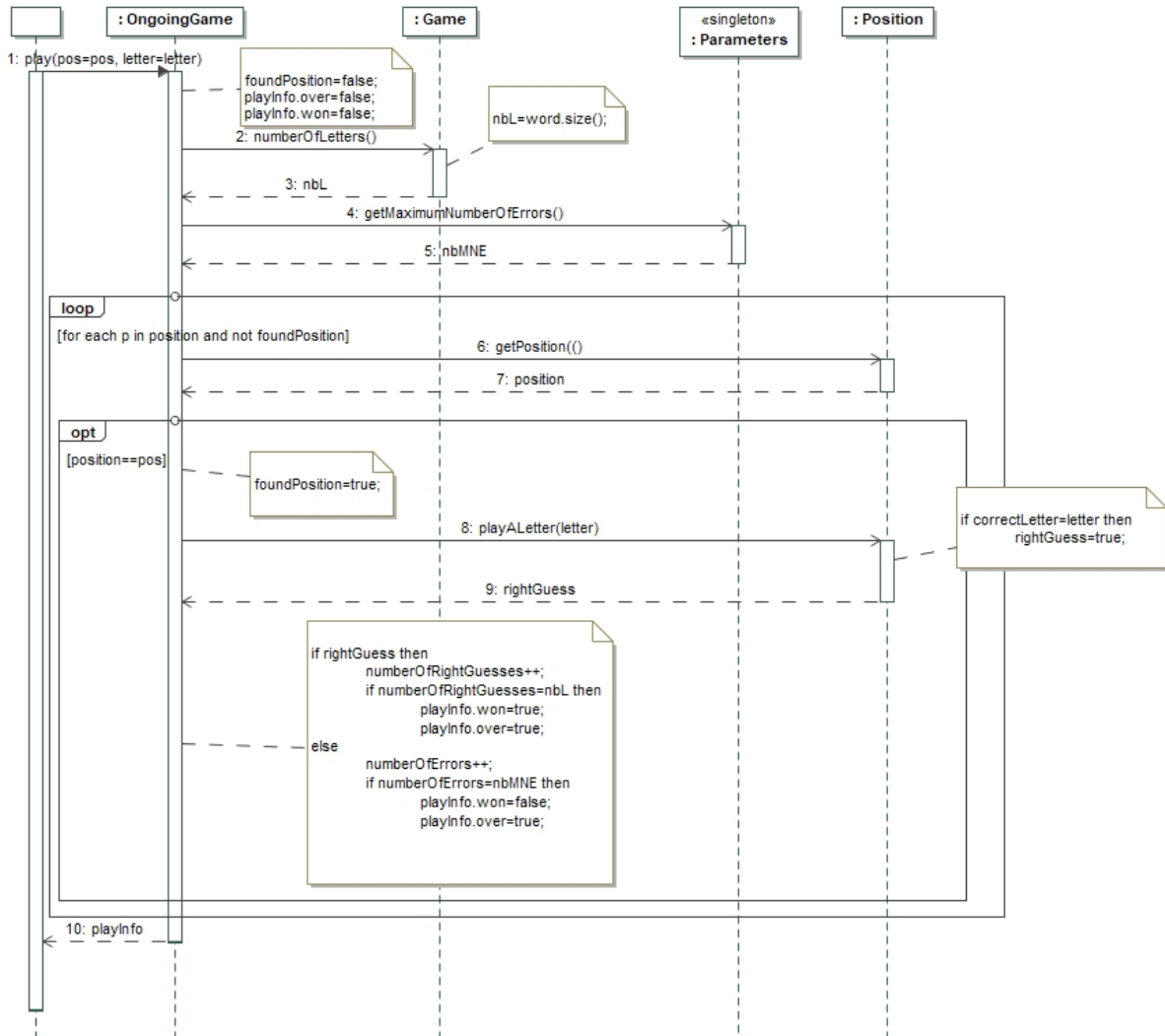
Examen 2020/21-2

Asignatura	Código	Fecha	Hora inicio
Análisis y diseño con patrones	75.586	19/6/2021	09:30



Examen 2020/21-2

Asignatura	Código	Fecha	Hora inicio
Análisis y diseño con patrones	75.586	19/6/2021	09:30



- b. Se ha aplicado el patrón de diseño **Estado** (clase abstracta y las dos clases que heredan de la clase abstracta) para representar los dos estados de la partida (en juego o acabada). También se ha aplicado el patrón de asignación de responsabilidades **Experto** (para identificar las clases que son las responsables de hacer las consultas y tratamientos). Por ejemplo, la operación `playALetter(letter:Char)` la hemos asignado a la clase `Position` ya que es esta clase la que tiene la información de la posición para poder poner la letra y saber si se ha adivinado o no.

También hemos aplicado el patrón **Singleton** para representar la información de los parámetros de las partidas. A continuación disponéis del diagrama de clases de diseño con las operaciones utilizadas en el diagrama de secuencia. Aunque no se pedía en el enunciado, se ha incluido el diagrama de clases para facilitar que se pueda entender mejor los diagramas de secuencia.

Examen 2020/21-2

Asignatura	Código	Fecha	Hora inicio
Análisis y diseño con patrones	75.586	19/6/2021	09:30

